

ASSIGNMENT WEEK-8

RAG Q&A chatbot:-

```
from google.colab import files
```

```
uploaded = files.upload()
```

```
import zipfile
```

```
import os
```

```
# Define paths
```

```
zip_path = "/content/archive (2).zip"
```

```
extract_path = "/content/loan_data"
```

```
# Extract
```

```
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
```

```
    zip_ref.extractall(extract_path)
```

```
# Confirm files inside
```

```
os.listdir(extract_path)
```

```
 ['Training Dataset.csv', 'Test Dataset.csv', 'Sample_Submission.csv']
```

```
import pandas as pd
```

```
# Load the training dataset
```

```
df = pd.read_csv("/content/loan_data/Training Dataset.csv")
```

```
# Preview data
```

```
df.head()
```

```
documents = []
```

```

for index, row in df.iterrows():

    text = (

        f"Loan ID: {row['Loan_ID']} | Gender: {row['Gender']} | Married: {row['Married']} | Dependents: {row['Dependents']} | "

        f"Education: {row['Education']} | Self Employed: {row['Self_Employed']} | "

        f"Applicant Income: ₹{row['ApplicantIncome']} | Coapplicant Income: ₹{row['CoapplicantIncome']} | "

        f"Loan Amount: ₹{row['LoanAmount']} | Loan Term: {row['Loan_Amount_Term']} months | "

        f"Credit History: {row['Credit_History']} | Property Area: {row['Property_Area']} | "

        f"Loan Status: {row['Loan_Status']}"

    )

    documents.append(text)

```

Preview a few

```
for doc in documents[:3]:
```

```
    print(doc)
```

```

Loan ID: LP001002 | Gender: Male | Married: No | Dependents: 0 | Education: Graduate | Self Employed: No | Applicant Income: ₹5849 |
Loan ID: LP001003 | Gender: Male | Married: Yes | Dependents: 1 | Education: Graduate | Self Employed: No | Applicant Income: ₹4583 |
Loan ID: LP001005 | Gender: Male | Married: Yes | Dependents: 0 | Education: Graduate | Self Employed: Yes | Applicant Income: ₹3000

```

```

Coapplicant Income: ₹0.0 | Loan Amount: ₹nan | Loan Term: 360.0 months | Credit History: 1.0 | Property Area: Urban | Loan Status: Y
| Coapplicant Income: ₹1508.0 | Loan Amount: ₹128.0 | Loan Term: 360.0 months | Credit History: 1.0 | Property Area: Rural | Loan Status: N
| Coapplicant Income: ₹0.0 | Loan Amount: ₹66.0 | Loan Term: 360.0 months | Credit History: 1.0 | Property Area: Urban | Loan Status: Y

```

!pip install -q sentence-transformers faiss-cpu

```

_____ 31.3/31.3 MB 23.9 MB/s eta 0:00:00
_____ 363.4/363.4 MB 4.4 MB/s eta 0:00:00
_____ 13.8/13.8 MB 25.8 MB/s eta 0:00:00
_____ 24.6/24.6 MB 25.1 MB/s eta 0:00:00
_____ 883.7/883.7 kB 29.1 MB/s eta 0:00:00
_____ 664.8/664.8 MB 2.4 MB/s eta 0:00:00
_____ 211.5/211.5 MB 5.6 MB/s eta 0:00:00
_____ 56.3/56.3 MB 9.9 MB/s eta 0:00:00
_____ 127.9/127.9 MB 8.6 MB/s eta 0:00:00
_____ 207.5/207.5 MB 3.6 MB/s eta 0:00:00
_____ 21.1/21.1 MB 40.7 MB/s eta 0:00:00

```

```
from sentence_transformers import SentenceTransformer
```

```
import faiss
```

```
import numpy as np
```

```
# Load lightweight model
```

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
# Generate embeddings
```

```
embeddings = model.encode(documents)
```

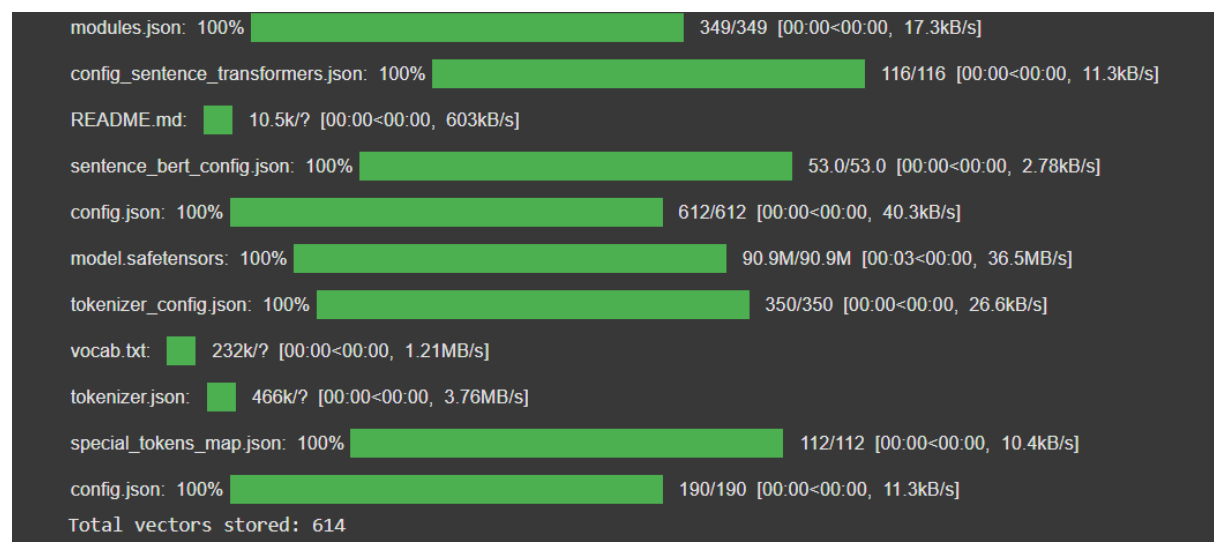
```
# Create FAISS index
```

```
dimension = embeddings.shape[1]
```

```
index = faiss.IndexFlatL2(dimension)
```

```
index.add(np.array(embeddings))
```

```
print(f"Total vectors stored: {index.ntotal}")
```



```
!pip install -q transformers
```

```
from transformers import pipeline
```

```
# Load a small text generation model
```

```
generator = pipeline("text-generation", model="sshleifer/tiny-gpt2") # fast + light
```

```
def ask_question(query, k=3):
```

```
    # Embed the query
```

```
    query_vec = model.encode([query])
```

```
# Search for top-k relevant docs
```

```
_, indices = index.search(np.array(query_vec), k)
```

```
# Get the top relevant rows
```

```
context = "\n".join([documents[i] for i in indices[0]])
```

```
# Prepare prompt
```

```
prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
```

```
# Generate response
```

```
result = generator(prompt, max_length=100, do_sample=True)[0]['generated_text']
```

```
print(result.replace(prompt, "Answer:"))
```

```
config.json: 100% ██████████ 662/662 [00:00<00:00, 16.1kB/s]
pytorch_model.bin: 100% ██████████ 2.51M/2.51M [00:00<00:00, 4.40MB/s]
tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 400B/s]
vocab.json: █ 899k/? [00:00<00:00, 1.60MB/s]
model.safetensors: 100% ██████████ 2.51M/2.51M [00:00<00:00, 4.11MB/s]
merges.txt: █ 456k/? [00:00<00:00, 3.84MB/s]
special_tokens_map.json: 100% ██████████ 90.0/90.0 [00:00<00:00, 1.88kB/s]
Device set to use cpu
```

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

tokenizer_config.json: 2.54k/? [00:00<00:00, 46.4kB/s]
spiece.model: 100% 792k/792k [00:01<00:00, 625kB/s]
tokenizer.json: 2.42M/? [00:00<00:00, 36.4MB/s]
special_tokens_map.json: 2.20k/? [00:00<00:00, 180kB/s]
config.json: 1.40k/? [00:00<00:00, 90.0kB/s]
model.safetensors: 100% 308M/308M [00:08<00:00, 59.6MB/s]
generation_config.json: 100% 147/147 [00:00<00:00, 8.77kB/s]

def ask_question(question):
    input_text = f"Question: {question} Context: {context_text}"
    inputs = tokenizer(input_text, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=100)
    print("Answer:", tokenizer.decode(outputs[0], skip_special_tokens=True))

```

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

```
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
```

```
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")
```

```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

```

```
ask_question("What is the maximum loan amount in the dataset?")
```

➡ Answer: 5849